

Aula 10 — Robustez e Persistência

Reinicialização, cão de guarda, baixo consumo e memória não volátil

Microcontroladores — DENE/UFMT

Objetivos da aula

- Enumerar as fontes de reinicialização do dispositivo e diagnosticar, em execução, qual delas ocorreu.
- Explicar a função da proteção contra subtensão e por que ela é indispensável em sistemas alimentados pela rede.
- Empregar corretamente o cão de guarda, reconhecendo o que ele protege e o que não protege.
- Descrever os modos de baixo consumo e as fontes capazes de despertar o dispositivo.
- Gravar e ler a memória não volátil interna, respeitando a sequência de desbloqueio, o tempo de escrita e o limite de ciclos.

1 Por que esta aula existe

Um sistema embarcado opera sem operador, por meses, sujeito a interferência elétrica, oscilações de alimentação e defeitos de software que não se manifestaram em bancada. As duas perguntas desta aula são práticas: **como o sistema se recupera quando algo dá errado** e **como ele preserva informação quando é desligado**.

2 Reinicialização

Definição — reinicialização

Evento que leva o dispositivo a um estado inicial conhecido — registradores em valores padrão, contador de programa no endereço zero — a partir do qual a execução recomeça.

Fonte	Quando ocorre	Significado para o projeto
Energização	Ao aplicar alimentação	Início normal
Subtensão	Alimentação abaixo de um limiar configurável	Rede instável, fonte subdimensionada ou carga pesada comutando
Pino externo	Sinal aplicado ao pino de reinicialização	Botão de reset ou circuito supervisor
Cão de guarda	O programa deixou de alimentá-lo no prazo	O firmware travou: é um diagnóstico, não um acidente
Estouro da pilha	Aninhamento de chamadas além da capacidade	Recursão ou cadeia de chamadas profunda demais
Instrução dedicada	O próprio programa solicita	Reinício controlado

Tabela 1: Fontes de reinicialização.

Observação

O dispositivo mantém indicadores que permitem descobrir, **em execução**, qual fonte causou o reinício. Lê-los logo no início do programa e registrá-los — na memória não volátil ou na telemetria — transforma “o equipamento reiniciou sozinho” em informação acionável. Sem esse registro, um sistema em campo que reinicia esporadicamente é praticamente indagnosticável.

2.1 Proteção contra subtensão

Abaixo de certa tensão, o dispositivo não deixa de funcionar de forma limpa: ele funciona **mal**. A memória pode ser lida incorretamente, uma escrita na memória não volátil pode ser interrompida no meio, e instruções podem ser decodificadas de forma errada.

Atenção

A faixa perigosa não é “sem alimentação”, é “alimentação insuficiente” — e ela é atravessada em toda energização e em todo desligamento. A proteção contra subtensão mantém o dispositivo em reinicialização enquanto a tensão estiver abaixo do limiar, garantindo que ele só execute em condições válidas. Em qualquer sistema alimentado pela rede, ela deve estar habilitada.

3 O cão de guarda

Definição — cão de guarda

Contador independente do núcleo, com oscilador próprio, que provoca uma reinicialização ao transbordar. O programa deve reiniciá-lo periodicamente — “alimentá-lo” — por meio de uma instrução dedicada. Se deixar de fazê-lo, o sistema é reiniciado.

A lógica é simples: um programa que executa normalmente passa pelo ponto de alimentação com regularidade. Um programa travado, preso num laço inesperado ou aguardando um sinal que nunca chega, não passa — e o cão de guarda o reinicia.

Atenção

Onde alimentar é uma decisão de projeto, não de conveniência. A alimentação deve ocorrer em **um único ponto**, no laço principal, depois de as tarefas essenciais terem sido executadas. Alimentá-lo dentro de um tratador de interrupção anula sua utilidade: as interrupções continuam ocorrendo mesmo com o laço principal travado, e o cão de guarda passa a atestar que o temporizador funciona — o que ninguém questionava.

Código

```
for (;;) {  
    if (!g_tique) {  
        continue;  
    }  
    g_tique = 0;
```

```

    executar_tarefas();

    CLRWD();      /* unico ponto de alimentacao, ao fim do ciclo */
}

```

Observação

Vale ser claro sobre os limites. O cão de guarda **não** protege contra lógica errada que continue rodando, contra um controlador que aqueça a planta indefinidamente com o laço funcionando, nem contra corrupção de dados. Ele detecta uma classe específica de falha — a parada de progresso — e a trata com o remédio mais bruto disponível. É valioso e é modesto; anunciá-lo como garantia de confiabilidade é engano.

Atenção

Um sistema que reinicia periodicamente sem motivo aparente tem, quase sempre, o cão de guarda habilitado na palavra de configuração e nunca alimentado — frequentemente porque o estudante copiou um bloco de configuração de outro projeto. É a hipótese a verificar antes de qualquer outra, e liga-se diretamente à tabela de bits de configuração do encontro 1.

4 Modos de baixo consumo

A instrução de suspensão desliga o oscilador principal e para a execução. O consumo cai de miliampères para microampères, e o dispositivo permanece assim até que um evento o desperte.

Fonte de despertar	Observação
Interrupções externas	Resposta imediata a um evento em pino
Mudança de estado em porta	Útil para teclado
Cão de guarda	Despertar periódico, sem oscilador principal
Comparador	Opera sem clock; vide encontro 5
Temporizador com cristal próprio	Base de tempo mantida durante o repouso

Tabela 2: Eventos capazes de despertar o dispositivo.

Atenção

Ao despertar, a execução prossegue na instrução **seguinte** à de suspensão — e, se a interrupção correspondente estiver habilitada, o tratador é executado antes. Programas escritos supondo que o despertar reinicia o dispositivo se comportam de maneira inesperada. Convém também lembrar que periféricos dependentes do oscilador principal — conversor, modulação, comunicação serial — param durante o repouso.

Observação

O termostato do laboratório é alimentado pela rede e não se beneficia disso. A técnica é apresentada porque é o principal argumento comercial dos microcontroladores em produtos a bateria: um sensor que dorme 99,9% do tempo, desperta por temporizador, mede,

transmite e volta a dormir opera anos com uma pilha. Todo o projeto de firmware desses produtos gira em torno de minimizar o tempo acordado.

5 Memória não volátil interna

O dispositivo dispõe de 256 bytes de memória não volátil de dados, gravável pelo próprio programa e preservada com o equipamento desligado — o lugar natural para o valor desejado de temperatura configurado pelo operador.

Característica	Ordem de grandeza
Capacidade	256 bytes
Tempo de escrita de um byte	Alguns milissegundos
Tempo de leitura	Um ciclo de instrução
Ciclos de escrita suportados	Centenas de milhares por posição
Retenção de dados	Décadas

Tabela 3: Características da memória não volátil. Confirmar na folha de dados.

Atenção

A assimetria entre leitura e escrita é enorme — da ordem de mil vezes — e define como o recurso deve ser usado: **ler à vontade, escrever raramente**. Uma escrita bloqueante de alguns milissegundos no meio do laço principal tem o mesmo custo de uma atualização completa de display.

5.1 A sequência de desbloqueio

A escrita não é uma atribuição: exige uma sequência específica de dois valores gravados num registrador de controle, imediatamente antes do comando de escrita.

Observação

A sequência existe para impedir escritas acidentais. Um programa desgovernado — ou um transiente elétrico — pode escrever num registrador por acaso; a chance de reproduzir exatamente dois valores específicos, na ordem certa e sem nada entre eles, é desprezível. É uma proteção contra a própria falha do firmware, coerente com o tema desta aula.

Atenção

A exigência de que **nada** ocorra entre os dois valores tem consequência direta: uma interrupção atendida no meio da sequência a invalida, e a escrita falha silenciosamente. As interrupções devem ser desabilitadas durante a sequência e reabilitadas em seguida — uma seção crítica, no sentido exato do encontro 9.

5.2 Desgaste

Cada posição suporta um número finito de escritas. Gravar o valor desejado a cada ciclo do laço principal esgotaria a vida útil da posição em pouco tempo.

Código

```
/* Grava apenas quando o valor mudou e depois de estabilizado. */
static int16_t gravado = 0;
static uint16_t ms_desde_mudanca = 0;

void persistir(int16_t alvo)          /* chamada a cada 1 ms */
{
    if (alvo == gravado) {
        ms_desde_mudanca = 0;
        return;
    }

    if (++ms_desde_mudanca < 5000u) { /* espera 5 s de estabilidade */
        return;
    }

    eeprom_escrever_16(ENDERECO_ALVO, alvo);
    gravado = alvo;
    ms_desde_mudanca = 0;
}
```

Observação

As duas proteções são cumulativas e vale nomeá-las: escrever apenas quando o valor **mudou**, e apenas depois de ele ter **permanecido estável** por algum tempo. A segunda evita gravar cada passo intermediário enquanto o operador percorre valores no teclado — que, sem ela, produziria dezenas de escritas para uma única alteração real.

6 Transposição

Duas diferenças relevantes nas plataformas de 32 bits. A primeira: a maioria **não possui** memória não volátil de dados separada. A persistência é feita reservando um setor da memória de programa e emulando o comportamento em software — o que envolve apagar blocos inteiros e distribuir o desgaste entre posições, tarefa entregue a bibliotecas específicas. O recurso simples usado aqui é, nesse aspecto, mais conveniente que o das plataformas modernas.

A segunda: costuma haver **dois** cães de guarda, um independente com oscilador próprio e outro com janela de tempo, que exige a alimentação nem cedo nem tarde demais — capaz, portanto, de detectar também um programa que enlouqueceu e passou a alimentá-lo rápido demais.

7 Exercícios

Exercício 10.1

Um equipamento em campo reinicia algumas vezes por dia, sem padrão aparente. Descreva um procedimento de diagnóstico baseado nos indicadores de fonte de reinicialização, indicando o que cada resultado possível sugere como causa.

Exercício 10.2

Explique por que alimentar o cão de guarda dentro do tratador de interrupção do temporizador anula sua função. Em seguida, descreva uma falha real de firmware que essa configuração deixaria passar despercebida.

Exercício 10.3

Um sistema grava o valor desejado na memória não volátil a cada 100 ms. Supondo o limite de ciclos da tabela desta aula, calcule em quanto tempo a posição atingiria esse limite, e compare com a vida útil pretendida de um equipamento industrial.

Exercício 10.4

Justifique tecnicamente por que as interrupções devem ser desabilitadas durante a sequência de desbloqueio da escrita, e estime o impacto dessa seção crítica sobre a latência do sistema, considerando os tempos desta aula.

Exercício 10.5

Um colega propõe habilitar o cão de guarda com o menor período disponível, argumentando que “quanto mais rápido detectar o travamento, melhor”. Analise a proposta apontando pelo menos dois riscos concretos, e proponha um critério objetivo para escolher o período.